```
        format.json { render :show, status: :created,
          location: @order }
      else
        format.html { render :new, status: :unprocessable_entity }
        format.json { render json: @order.errors,
          status: :unprocessable_entity }
      end
    end
  end
```

And we need to update `shipped` as we did for `received`:

rails80/depot_qa/app/mailers/order_mailer.rb

```
  def shipped(order)
    @order = order

    mail to: order.email, subject: "Pragmatic Store Order Shipped"
  end
```

Now we have enough of the basics in place that you can place an order and have a plain email sent to yourself, assuming you didn't disable the sending of email in development mode. Let's spice up the email with a bit of formatting.

Delivering Multiple Content Types

Some people prefer to receive email in plain-text format, while others like the look of an HTML email. Rails supports this directly, allowing you to send email messages that contain alternative content formats, allowing users (or their email clients) to decide which they'd prefer to view.

In the preceding section, we created a plain-text email. The view file for our `received` action was called `received.text.erb`. This is the standard Rails naming convention. We can also create HTML-formatted emails.

Let's try this with the order-shipped notification. We don't need to modify any code—we simply need to create a new template: